# Frame Generation and Upscaling

## Scope

This document defines the rendering policy for:

- FSR1 spatial upscaling;
- FSR2 temporal upscaling;
- DLSS temporal upscaling on NVIDIA;
- FSR3 frame generation;
- DLSS frame generation on NVIDIA;
- custom frame generation.

XeSS upscaling and XeSS frame generation are not part of the product. MetalFX remains a reserved serialized value with no adapter.

It also documents the interaction with HUD, render scale, OIT, static layer bake, alpha cutout and draw-call batching.

All listed technologies are disabled by default. A project must explicitly opt in. A capability check must also pass before an option appears in the user interface.

FSR1, FSR2, FSR3 frame generation, DLSS/DLSS frame generation and custom frame generation have engine-side contracts. Vendor backends load their runtime DLLs dynamically; missing runtime or unsupported hardware falls back without crashing.

## Terminology

Spatial upscaling reconstructs a display image from a lower-resolution image without temporal history. FSR1 and bilinear filtering belong to this group.

Temporal upscaling uses color, depth, motion vectors and previous frames. FSR2 and DLSS belong to this group.

Frame generation inserts an additional display frame between two rendered frames. It does not replace upscaling. It operates after the scene has been upscaled to display resolution.

HUD means screen-space UI, text, menus, cursor, subtitles and accessibility

overlays. HUD is not part of the game scene for frame generation.

## Render pipeline

The supported order is:

1. render the game scene into a scene target;
2. optionally upscale the scene to display resolution;
3. optionally generate intermediate display frames;
4. render HUD into a separate native-resolution target;
5. composite the generated scene with HUD;
6. present the composite.

HUD must not enter motion vectors, depth, reactive masks or frame-generation

history. HUD is composited after upscaling and after frame generation.

The minimum data required by the pipeline is:

- scene color;
- depth;
- camera motion vectors;
- per-sprite motion vectors;
- reactive mask for transparency, particles and OIT;
- previous camera;
- history validity;
- output extent and render extent.

## Custom frame generation

The custom frame generator is a portable stage that runs after the selected

upscaler. It is not tied to a GPU vendor.

Its input contract is:

- upscaled scene color at display resolution;
- depth at render resolution;
- motion vectors at render resolution;
- reactive mask;
- camera delta;
- jitter and history state;
- frame timing.

Its output is a display-resolution scene frame. HUD is not an output of the

generator.

The processing stages are:

7. validate history and motion vectors;
8. reproject the previous scene;
9. detect disocclusion from depth;
10. apply the reactive mask to unstable pixels;
11. interpolate the scene;
12. blend with the current real frame;
13. mark the frame as generated or real.

The generator must support a fail-safe path. If motion confidence is low, if the

scene was reset, if the frame time is unstable or if the render extent changed,

the generator returns the real frame instead of an interpolated frame.

## Settings

Project policy is stored in `project.toml`:

```
[render]
render_scale = 1.0
upscale_policy = "auto"          # auto | universal | off
frame_generation_policy = "auto" # auto | universal | off
allow_temporal_upscale = true
allow_frame_generation = true
```

Player settings are stored separately:

```
[graphics]
render_scale = 1.0
upscale_policy = "auto"
frame_generation_policy = "auto"
```

The player chooses a policy, not a vendor:

- Auto: DLSS on NVIDIA, otherwise FSR2, otherwise FSR1; DLSS FG on NVIDIA,

otherwise FSR3 FG, otherwise custom;

- Universal: FSR2/FSR1 and FSR3 FG/custom;
- Off: bilinear and no frame generation.

The vendor enums remain in project/user files for cross-platform projects.

The UI filters unavailable values through capabilities. It does not present a

dead control.

## Capability checks

Capabilities are reported by the RHI backend:

```
struct UpscalerCaps
{
    bool nearest;
    bool bilinear;
    bool bicubic;
    bool fsr1;
    bool fsr2;
    bool dlss;
    bool motionVectors;
    bool frameGeneration;
};
```

A backend is selectable only when all required flags are true:

- FSR1: `fsr1`;
- FSR2: `fsr2 && motionVectors`;
- DLSS: Vulkan + Windows + NVIDIA + NGX runtime + motion vectors;
- FSR3 FG: FidelityFX runtime + motion vectors;
- DLSS FG: NVIDIA + DLSS-G feature + motion vectors;
- custom FG: temporal resolve + motion vectors + HUD separation.

Optional vendor build flags:

```
FBY_WITH_DLSS=OFF
FBY_WITH_FIDELITYFX=OFF
```

FSR1/FSR2/custom FG are core and runtime-gated. When a vendor backend is not compiled or its runtime is missing, its capability is false and settings fall back. XeSS has been removed; MetalFX is reserved without an adapter.

## Optional vendor SDKs

The DLSS and FidelityFX SDKs are intentionally not committed to the engine repository. They are large, vendor-licensed and optional; a checkout without them still builds the portable FSR1/FSR2/custom paths. The `SetupScripts/Windows/SetupUpscalers.*`, `SetupScripts/Linux/SetupUpscalers.*` or `SetupScripts/macOS/SetupUpscalers.*` helpers verify or download a local SDK after the license is accepted. A developer who needs a vendor backend installs the SDK locally and points CMake at it:

```
# NVIDIA DLSS, Windows + Vulkan + NVIDIA only
cmake -S . -B build -DFBY_WITH_DLSS=ON -DFBY_DLSS_ROOT="C:/SDK/NVIDIA-DLSS"
```

```
# AMD FidelityFX, FSR2/FSR3 adapter
cmake -S . -B build -DFBY_WITH_FIDELITYFX=ON \
      -DFBY_FIDELITYFX_ROOT="C:/SDK/FidelityFX/Kits/FidelityFX"
```

The expected layouts are:

- DLSS: `include/nvsdk_ngx.h` and the NGX runtime DLLs;
- FidelityFX: `api/include/ffx_api.h`, `upscalers/`, `framegeneration/`, `backend/vk/` and `signedbin/amd_fidelityfx_upscaler_vk.dll`.

`FBY_DLSS_ROOT` and `FBY_FIDELITYFX_ROOT` are cache paths, so a developer can keep the SDK outside the repository. A missing SDK is a configuration warning: the vendor backend is left disabled and the configure completes, so the engine and editor still build and runtime selection falls back. Pass `-DFBY_REQUIRE_VENDOR_SDKS=ON` when a CI job must fail on a missing SDK. The FidelityFX DLL is copied next to the executable; DLSS loads its NGX runtime dynamically from the driver/SDK search path. Never copy an SDK into a source commit; the default `third_party/dlss` and `third_party/fidelityfx` locations are ignored and are only local SDK roots.

## Mutual exclusion

Exactly one upscaler and at most one frame generator may be active.

Valid combinations:

- native scene + custom frame generation;
- FSR1 + custom frame generation;
- FSR2 + custom frame generation, after explicit testing;
- DLSS + custom frame generation, after explicit testing;
- native scene + vendor frame generation.

Invalid combinations:

- FSR2 and DLSS simultaneously;
- two frame generators simultaneously;
- DLSS Frame Generation and custom frame generation;
- FSR3 Frame Generation and custom frame generation;
- HUD rendered into the frame-generation input.

The engine resolves invalid combinations before rendering. It does not start two

backends and hope for the best. The project setting has priority over the user setting. The capability check has priority over both.

## Compatibility matrix

| Upscaler | Custom frame generation | Vendor frame generation |
|---|---|---|
| Native | Supported | Supported, vendor only |
| FSR1 | Recommended | Supported, vendor only |
| FSR2 | Allowed after testing | Not with custom |
| DLSS | Allowed after testing | Not with custom |
| Bilinear/Bicubic | Supported | Supported, vendor only |

FSR1 plus custom frame generation is the preferred portable combination because FSR1 has no temporal history of its own. FSR2 or DLSS plus custom frame generation introduces two temporal stages and must be evaluated for ghosting.

## HUD rules

HUD is rendered at native resolution. A separate HUD render scale may be used, but it is independent from the game render scale.

HUD target modes:

- `Direct`: UI is drawn into the swapchain. This is the fastest path and does not allocate an offscreen HUD target.
- `Offscreen`: UI is drawn into a HUD target and composited after frame generation.
- `Auto`: Direct when frame generation is off, Offscreen when it is on.

`hud.render_scale`, `hud.upscale` and `hud.visible` apply to the offscreen path. The game settings expose the target mode; the project may restrict it.

`Direct` with active frame generation disables generation rather than placing text into the temporal history.

HUD must never be passed to FSR2, DLSS, vendor frame generation or custom frame generation. Text quality is preserved only when HUD remains outside the temporal pipeline.

## Collision resolution

Collisions are resolved in this order:

14. project policy;
15. build availability;
16. device capability;
17. user choice;
18. fallback.

Rules:

- a project that disables temporal upscaling hides FSR2 and DLSS;
- a device without motion vectors disables every temporal path;
- a device without a frame-generation backend disables the generator;
- vendor frame generation and custom frame generation cannot be active together;
- selecting a new upscaler resets temporal history and frame-generation history;
- changing render scale or display extent resets both histories;
- changing HUD mode does not affect scene history;
- OIT, particles and transparent effects must set the reactive mask;
- HUD changes must not invalidate scene history.

If a selected backend disappears after a device reset, the engine falls back to

native rendering and reports the reason in diagnostics.

## Artifacts

| Artifact | Cause | Mitigation |
|---|---|---|
| Blockiness | Low render scale with nearest filtering | Bilinear, bicubic, FSR1, higher render scale |
| Blur | Bilinear or low-quality spatial upscale | FSR1, bicubic, higher render scale |
| Ghosting | Wrong motion vectors or stale history | Reset history, validate vectors, reactive mask |
| Smearing | Transparency, particles or OIT in temporal history | Reactive mask, disable generation for the effect |
| Edge crawl | No jitter or incorrect depth | Correct jitter, depth and motion vectors |
| Text ghosting | HUD inside frame generation | Keep HUD native and composite after |
| Input latency | Generated frames presented late | Frame pacing, latency budget, disable on low FPS |
| Double temporal ghosting | FSR2 or DLSS plus custom frame generation | Test, or use FSR1 plus generator |
| Pixel-art wobble | Subpixel generation on integer art | Disable generation for pixel-art preset |

## Best choices by game type

Pixel art, integer-scaled sprites: use nearest filtering and no frame generation.

Use a render scale of 1.0 when the target GPU allows it.

Painted 2D with camera movement: FSR1 is the safe portable choice. Custom frame generation may be enabled when motion vectors are correct and HUD is separate.

Fast action with transparent effects: FSR1 plus custom frame generation only after reactive-mask validation. Disable generation if particles smear.

HUD-heavy games: keep HUD native. Do not enable frame generation if the HUD cannot be separated from the scene.

Static backgrounds and slow cameras: static bake plus FSR1 provides the largest gain with the fewest temporal artifacts.

Screenshots and benchmarks: set HUD to `hidden`, disable frame generation, and use native rendering for reproducible results.

## Other engine technologies

Render scale reduces the internal scene resolution. It is independent from HUD scale. A lower render scale reduces fill rate but requires an upscaler to restore display resolution.

OIT is an order-independent transparency path with accumulation and revealage targets. OIT pixels are reactive; frame generation must not treat them as stable scene history without a mask.

Static layer bake reuses a rendered scene while camera, scene and lights are unchanged. It is invalidated by camera movement, light changes, scene changes, post-processing changes and SDF changes.

Alpha cutout converts binary-alpha opaque art into a depth-tested path. It is safe for front-to-back rendering only when the cooked texture metadata marks the layer as opaque or cutout. Blended art must stay on the normal path.

Single-indirect opaque rendering combines compatible opaque batches into one indirect draw. It does not change pixel cost and does not replace upscaling.

## Verification checklist

- all backends default off;
- caps hide unsupported options;
- project policy has priority over user choice;
- one upscaler only;
- one frame generator only;
- HUD native and outside temporal history;
- reset on extent, scale, scene and camera discontinuity;
- reactive mask for OIT and particles;
- fallback to real frame on low confidence;
- pixel-art preset disables generation;
- diagnostics report the active backend and fallback reason.